



Wrocław
University
of Science
and Technology

Software Engineering

Project management and software development planning

Marcin Jodłowiec

October 15th, 2025

Agenda

- Project planning and scheduling
Why planning matters?
- Cost modeling and estimation techniques
- Risk analysis and mitigation
- Team roles and organization in software development

Why planning matters?

- ▶ Most projects are **not successful**.
- ▶ Evidence: **CHAOS 2015 (Standish Group)**
- ▶ Top 3 reasons for project failure:
 - ▶ Incomplete requirements and specifications,
 - ▶ Lack of user involvement,
 - ▶ Unrealistic schedules.
- ▶ Key point: **We need planning.**
 - ▶ Effective planning helps reduce the number of challenged and failed projects

	2011	2012	2013	2014	2015
SUCCESSFUL	29%	27%	31%	28%	29%
CHALLENGED	49%	56%	50%	55%	52%
FAILED	22%	17%	19%	17%	19%

Project planning

Project planning

Project planning is process of defining objectives, scope, resources, schedule and risks in order to **guide the project toward successful completion**.

Key activities:

- ▶ Defining project scope and objectives,
- ▶ Identifying tasks, dependencies and milestones,
- ▶ Estimating time, cost and effort,
- ▶ Defining required resources,
- ▶ Assigning responsibilities and roles,
- ▶ Risk analysis, assessment and mitigation planning.

Software project scope

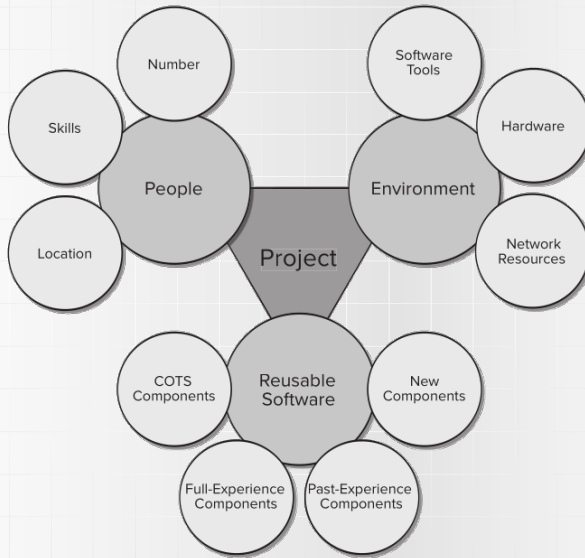
Definition

Software scope defines the **boundaries and goals** of a software project – what will be delivered and what will not. It must be **unambiguous, bounded, and measurable**.

Key elements of software scope:

- ▶ **Context:** How does the software fit into the larger system or business process? What external constraints affect it?
- ▶ **Information objectives:** What customer-visible data objects are produced or used? What inputs and outputs define the system's functionality?
- ▶ **Function and performance:** What transformations does the software perform, and what performance characteristics (e.g., speed, scalability, memory limits) must be met?

Project resources



Project scheduling

Project scheduling

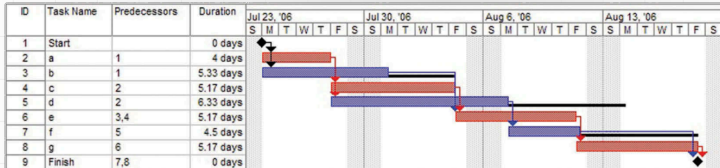
Project scheduling is the process of organizing, sequencing, and allocating time for project tasks to ensure timely completion.

- ▶ Determines **when** each task should start and finish.
- ▶ Helps identify dependencies between activities.
- ▶ Supports monitoring of project progress.
- ▶ Enables detection of potential bottlenecks and delays.

Common techniques:

- ▶ Gantt charts,
- ▶ Critical Path Method (CPM).
- ▶ PERT (Program Evaluation and Review Technique),

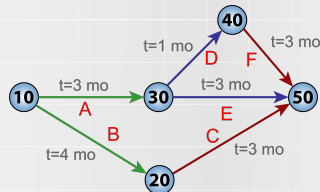
Gantt chart example



- ▶ Tasks are represented as horizontal bars.
- ▶ Length of each bar indicates task duration.
- ▶ Overlapping bars indicate concurrent tasks.
- ▶ Dependencies can be shown with arrows.
- ▶ Milestones are often marked with diamonds.
 - ▶ Finish-to-start (FS),
 - ▶ Start-to-start (SS),
 - ▶ Finish-to-finish (FF),

Critical Path Method (CPM) and PERT

- ▶ CPM is a project modeling technique used to identify the sequence of crucial tasks that determine the minimum project duration.
- ▶ Key concepts:
 - ▶ **Activity:** A task or work item in the project.
 - ▶ **Event:** A point in time marking the start or end of an activity.
 - ▶ **Network diagram:** A graphical representation of activities and their dependencies.
 - ▶ **Critical path:** The longest path through the network diagram, determining the shortest possible project duration.
- ▶ PERT – is a statistical tool used to analyze and represent the tasks involved in completing a project, especially when there is uncertainty in task durations. PERT chart example:



Parkinson's Law

"Work expands so as to fill the time available for its completion."

– C. Northcote Parkinson, 1955

Definition

- ▶ Parkinson's Law states that tasks tend to consume all the time that has been allocated to them.
- ▶ In project management, this means that if a team is given two weeks for a task, the work will usually expand to take the full two weeks – even if it could have been done faster.

Implications for Software Projects

- ▶ Overly generous schedules lead to inefficiency and procrastination.
- ▶ Work expands with added analysis, unnecessary refinements, and meetings.
- ▶ The law explains why traditional projects often suffer from schedule overruns.

Aids: time boxing, agile iterations.

Software cost and effort estimation

- ▶ Many variables: complexity, team skill, technology, environment, requirements volatility, etc.
- ▶ People do fail.
- ▶ Still, we need to try somehow.
- ▶ Estimation techniques:
 - ▶ delay estimation (the more time passes, the more accurate it gets),
 - ▶ estimate by analogy (compare to similar past projects),
 - ▶ **decompose and estimate** (break down into smaller tasks and estimate each),
 - ▶ Work Breakdown Structure (WBS),
 - ▶ **use empirical models** (use historical data and statistical methods).

Using the empirical models for estimation I

- ▶ **Approach:** Empirical estimation models use **regression analysis** to establish relationships between project size and effort.
- ▶ **Typical estimation variables:**
 - ▶ **LOC (Lines of Code)** – a measure of software size based on the number of lines written,
 - ▶ **FP (Function Points)** – a measure of functionality delivered, independent of programming language.
- ▶ **Typical model form:**

$$E = A + B \times (e_v)^C$$

where:

- ▶ E – effort (in person-months),
 - ▶ e_v – estimation variable (either LOC or FP),
 - ▶ A, B, C – empirically derived constants.
- ▶ **Project adjustment factors:**

Using the empirical models for estimation II

- ▶ problem complexity,
 - ▶ staff experience and capability,
 - ▶ development environment and tools.
- ▶ **General form:**

$$d = f(v_i)$$

where d is one of the estimated project attributes (effort, cost, or duration), and v_i are selected independent parameters (e.g., estimated LOC or FP).

- ▶ Empirical models complement decomposition-based estimation techniques and provide a data-driven approach grounded in observed project performance.

Using the empirical models for estimation III

Semantics of A,B and C

- ▶ A represents **effort bias** – the baseline effort required for a project of minimal size.
- ▶ B is a **productivity scaling factor** that adjusts the effort based on the size of the project.
 - ▶ B absorbs the effects of various project attributes, such as team experience, technology, and automation.
- ▶ C is an exponent that reflects the **non-linear relationship** between project size and effort, capturing the increasing complexity and overhead associated with larger projects.
 - ▶ $C \approx 1$ indicates a linear relationship (doubling size doubles effort),
 - ▶ $C > 1$ indicates superlinear growth (complexity of communication, integration, tests, management etc. grows faster than size increase),
 - ▶ $C < 1$ indicates the sublinear growth (component-based development, code reuse, frameworks, tools etc. help reduce effort growth).

Estimation for Agile Development I

- ▶ In agile projects, requirements are expressed as **user stories**.
- ▶ Estimation is performed for each software increment (typically 3–6 weeks).
- ▶ The process follows a **decomposition-based approach**:
 1. Each **user story** (a mini use case) is considered separately for estimation.
 2. The user story is decomposed into a set of software engineering **tasks**.
 3. Each task is estimated separately based on:
 - ▶ historical data,
 - ▶ an empirical model (e.g., COCOMO),
 - ▶ or expert experience (e.g., *planning poker*).
 4. Alternatively, the **volume** of the story can be expressed in:
 - ▶ LOC (Lines of Code),

Estimation for Agile Development II

- ▶ FP (Function Points),
 - ▶ or other measures (e.g., use case count).
- 5. Task estimates are summed to form the total effort for the user story.
- 6. Efforts for all user stories in a given increment are summed to estimate total effort for that increment.
- ▶ **Purpose of agile estimation:**
 - ▶ Ensure that the number of user stories planned for an increment fits available resources.
 - ▶ Establish a baseline for allocating effort as the increment progresses.

Planning Poker technique I

- **Purpose:** collaborative technique for estimating the **effort or complexity** of user stories in agile projects.

How it works

1. Each team member gets a deck of cards with values (e.g., Fibonacci series: 0, 1, 2, 3, 5, 8, 13, 20, 40, 100).
2. The **Product Owner** presents a user story and answers questions.
3. Each participant privately selects a card representing their estimate.
4. All cards are revealed **simultaneously** to avoid bias.
5. Team members discuss the differences, focusing on highest and lowest estimates.
6. The team repeats voting until a **consensus** is reached.

Planning Poker technique II

Key ideas

- ▶ Estimation is **relative**, not absolute: points reflect comparative effort, not hours.
- ▶ Promotes **team discussion, shared understanding, and balanced judgment**.
- ▶ Reduces anchoring and domination by more vocal team members.

Benefits

- ▶ Fast and engaging way to reach realistic estimates.
- ▶ Enhances team communication and shared ownership of planning.
- ▶ Builds a historical baseline for future velocity estimation.

Risk analysis and mitigation

- ▶ Risk is an uncertain future event or condition that, if it occurs, has a **positive or negative effect** on a project's objectives.
- ▶ Risk management is the process of **identifying, estimating, and controlling** risks.
- ▶ reactive vs proactive approach,
 - ▶ reactive – deal with problems as they arise,
 - ▶ proactive – identify and mitigate risks before they become issues → can reduce **technical debt**.

Software risks

- ▶ **Project risks** – threaten the project plan:
 - ▶ schedule delays, cost overruns,
 - ▶ staffing or resource shortages,
 - ▶ stakeholder or requirements problems.
- ▶ **Technical risks** – threaten product quality and feasibility:
 - ▶ design or implementation problems,
 - ▶ ambiguous specifications, integration issues,
 - ▶ obsolete or immature technology.
- ▶ **Business risks** – threaten product or organizational viability:
 - ▶ market risk – building something nobody wants,
 - ▶ strategic risk – misalignment with company goals,
 - ▶ sales, management, or budget loss.
- ▶ Risk categorization will not always work, some risks are unpredictable.

Risk mitigation, monitoring, and management I

Goal

Develop a proactive strategy for **identifying, reducing, and controlling risks** throughout the project lifecycle.

Key components:

- ▶ **Mitigation** – reduce the likelihood or impact of risk.
 - ▶ Example: prevent staff turnover by improving conditions, knowledge sharing, and backup assignments.
- ▶ **Monitoring** – observe risk indicators during the project.
 - ▶ Track team morale, resource stability, and early warning signs.
- ▶ **Management / Contingency** – act when risk becomes real.
 - ▶ Execute backup plans, reallocate resources, and ensure knowledge transfer.

Risk mitigation, monitoring, and management II

Pareto principle in risk management:

- ▶ Focus on the **20% of risks** that cause **80% of potential loss**.
- ▶ Apply detailed mitigation to the most critical few, monitor the rest.

Risk information sheet example

Risk information sheet			
Risk ID: P02-4-32	Date: 5 / 9 / 19	Prob: 80%	Impact: high
Description: Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.			
Refinement/context: Subcondition 1: Certain reusable components were developed by a third party with no knowledge of internal design standards. Subcondition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components. Subcondition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.			
Mitigation/monitoring: 1. Contact third party to determine conformance with design standards. 2. Press for interface standards completion; consider component structure when deciding on interface protocol. 3. Check to determine number of components in subcondition 3 category; check to determine if language support can be acquired.			
Management/contingency plan/trigger: RE computed to be \$20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly. Trigger: Mitigation steps unproductive as of 7 / 1 / 19.			
Current status: 5 / 12 / 19: Mitigation steps initiated.			
Originator: D. Gagne		Assigned: B. Laster	

Stakeholders

A **stakeholder** is any individual, group, or organization that can affect, or is affected by, the development or outcome of a software project. Some of them are people:

1. **Senior managers (Product owners)** – define business goals and constraints influencing the project.
2. **Project / Technical managers (Scrum masters, team leads)** – plan, motivate, organize, and coordinate the development team.
3. **Practitioners** – provide the technical expertise to design and implement the software.
4. **Customers and other stakeholders** – specify requirements and maintain an interest in the project's success.
5. **End users** – interact with the software after release and determine its real-world value.

Leaders vs. Managers in Software Projects

- ▶ **Leadership** and **management** are distinct but complementary roles in software projects.

Managers

- ▶ Focus on **planning**, **scheduling**, and **control**.
- ▶ Ensure tasks are completed **on time** and within **budget**.
- ▶ Emphasize **processes**, rules, and accountability.
- ▶ Tend to **direct** and **monitor** others.
- ▶ May monopolize decision-making authority.

Leaders

- ▶ Focus on **vision**, **motivation**, and **people**.
- ▶ Inspire commitment through **shared goals** and **trust**.
- ▶ Encourage **creativity** and **autonomy**.
- ▶ **Model the way** and lead by example.
- ▶ Celebrate success and foster **team spirit**.

Leaders vs. Managers in Software Projects



Belbin's Team Roles

Concept

Meredith Belbin identified nine key roles that individuals tend to adopt in teams. Each role contributes differently to team performance and balance.

- ▶ **Plant** – creative innovator, generates original ideas.
- ▶ **Monitor Evaluator** – logical, analyzes options objectively.
- ▶ **Coordinator** – clarifies goals, delegates effectively.
- ▶ **Resource Investigator** – explores external opportunities and contacts.
- ▶ **Implementer** – turns ideas into practical actions.
- ▶ **Completer Finisher** – detail-oriented, ensures quality.
- ▶ **Teamworker** – supportive, promotes harmony and collaboration.
- ▶ **Shaper** – dynamic, challenges inertia, drives progress.
- ▶ **Specialist** – brings in-depth expertise in a narrow field.

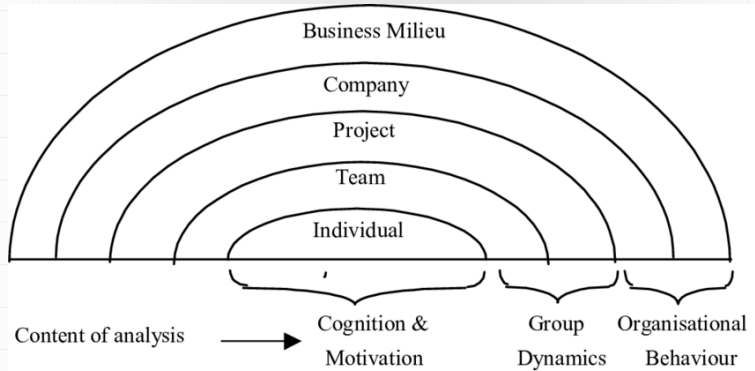
Belbin Roles in Software Teams

From psychology to software engineering

Belbin's roles can be mapped to typical functions within software teams. Understanding these mappings helps distribute responsibilities and leverage strengths.

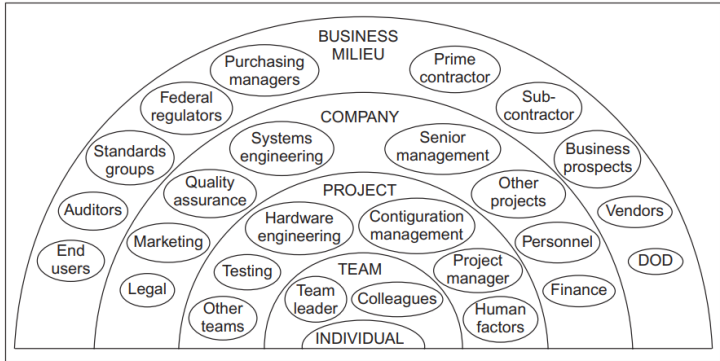
Belbin Role	Example in Software Team
Plant	System architect / idea person
Monitor Evaluator	QA analyst / reviewer
Coordinator	Scrum Master / project manager
Resource Investigator	Business analyst / product owner
Implementer	Senior developer / team lead
Completer Finisher	QA engineer / tester
Teamworker	Support engineer / DevOps / UX
Shaper	Tech lead / delivery driver
Specialist	Domain or technology expert

Psychology of Software Engineering (cont.)



The Layered Behavioural Model Of Software Development
(Source: Curtis et al., 1988)

Psychology of Software Engineering (cont.)



Layers of communication in behavioral model (Source: Curtis et al., 1988)

Formal roles in software development

- ▶ There is **no single, canonical list** of software development roles in the Software Engineering.
- ▶ **Formal roles** appear in specific frameworks and standards:
 - ▶ **Scrum Guide (2020)** – Product Owner, Developers, Scrum Master.
 - ▶ **ISO/IEC 12207** – Acquirer, Developer, Maintainer, Operator, User.
 - ▶ **PMBOK / PRINCE2** – Sponsor, Project Manager, QA, Stakeholders.
- ▶ Modern practice blends these into context-specific roles:
 - ▶ *Product roles*: Product Owner, Business Analyst, UX Designer.
 - ▶ *Engineering roles*: Architect, Developer, Tester, DevOps Engineer.
 - ▶ *Management roles*: Project Manager, Tech Lead, Scrum Master.

In practice, roles emerge from process models and team topology is derived from organization and process.

The Mythical Man-Month (Fred Brooks, 1975)

Essays on Software Engineering

Classic collection of insights from managing IBM's OS/360 project. Main message: **software development does not scale linearly with people or time.**

Key ideas:

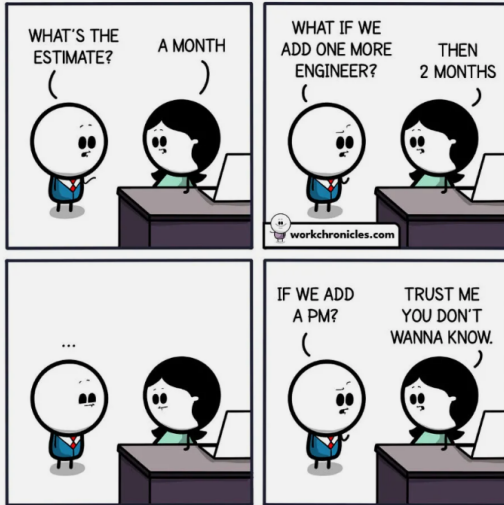
- ▶ Brook's Law: **Adding manpower to a late project makes it later.**
- ▶ Communication overhead grows exponentially with team size.
- ▶ Maintain **conceptual integrity** – small, coherent design leadership.
- ▶ Avoid the **second-system effect** – don't overengineer the next version.
- ▶ **"Build one to throw away"** – first system is a prototype.
- ▶ **No silver bullet:** no single tool or method will drastically improve productivity.

Famous quote

Nine women can't make a baby in one month.

— Frederick P. Brooks Jr.

It's all for today!



Comics about work. Made with ♥ & lots of coffee.
Get the comics straight to your Inbox. Join the Newsletter.

Work Chronicles
workchronicles.com